

Course Dashboard - Konzept

BESCHREIBUNG

Im Rahmen dieses Projekts soll ein Dashboard entwickelt werden, das einen schnellen Überblick über den eigenen Studienfortschritt ermöglicht. Dabei sollen unter anderem die aktuellen und bereits abgeschlossenen Module, Prüfungsleistungen und der bisherige Notendurchschnitt dargestellt werden. So soll auf einen Blick erkennbar sein, wie weit das Studium bereits fortgeschritten ist und ob die gesetzten Ziele erreicht werden.

ZIELE

Studienfortschritt überwachen

Der Fortschritt des Studiums soll jederzeit anhand der absolvierten Semester und Module erkennbar sein.

Notenziel überwachen

Der aktuelle Notendurchschnitt soll mit dem angestrebten Notendurchschnitt verglichen werden können.

Aktuellen Arbeitsstand überwachen

Es soll schnell erkennbar sein, welche Module bereits abgeschlossen wurden und welche Prüfungsleistungen noch ausstehen.

BENÖTIGTE DATEN

Studienfortschritt

- Anzahl der Semester
- aktuelles Semester
- Anzahl aller Module
- Anzahl abgeschlossener / offener Module
- Fortschritt in Prozent

Noten

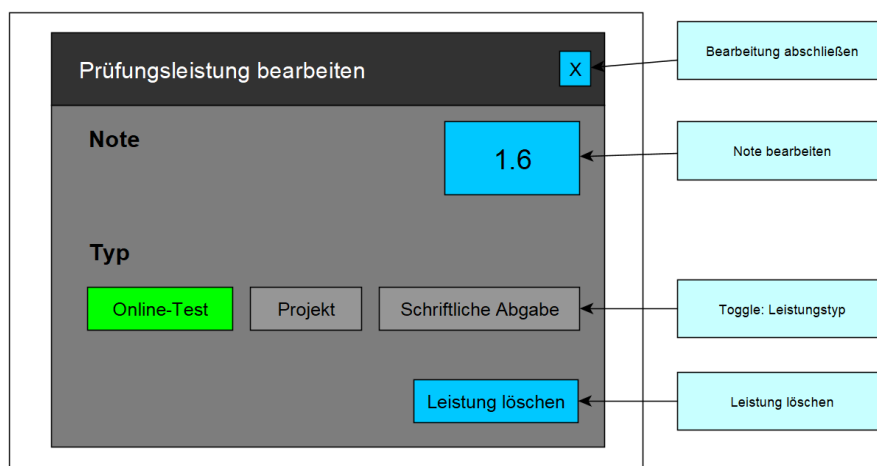
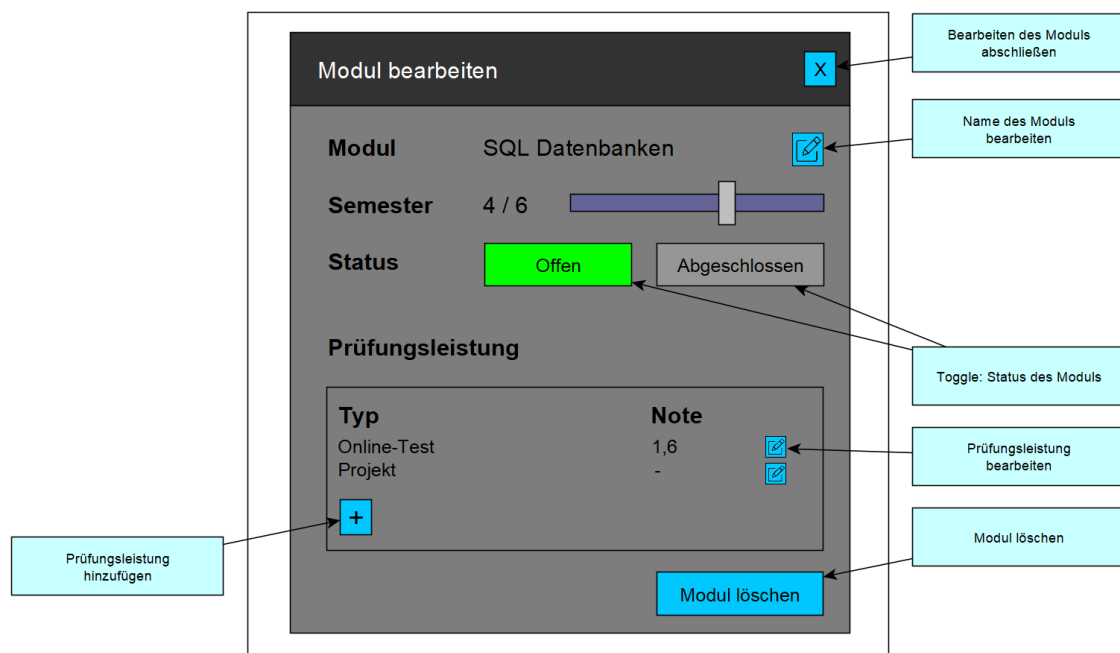
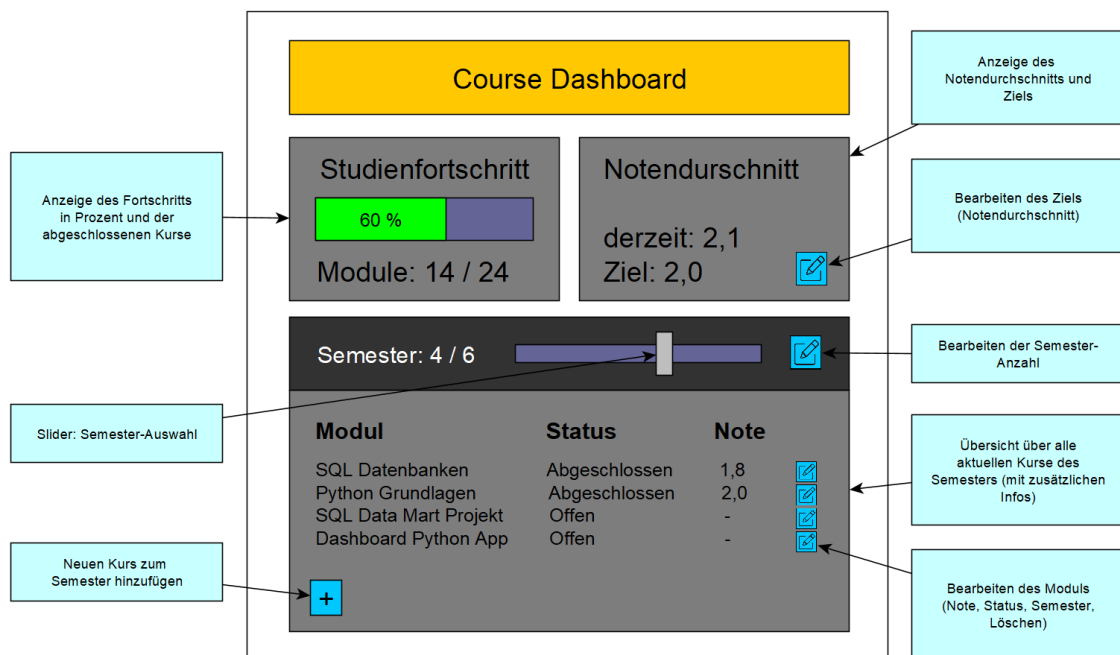
- einzelne Prüfungsleistungen
- erzielte Noten
- aktueller Notendurchschnitt
- angestrebter Notendurchschnitt

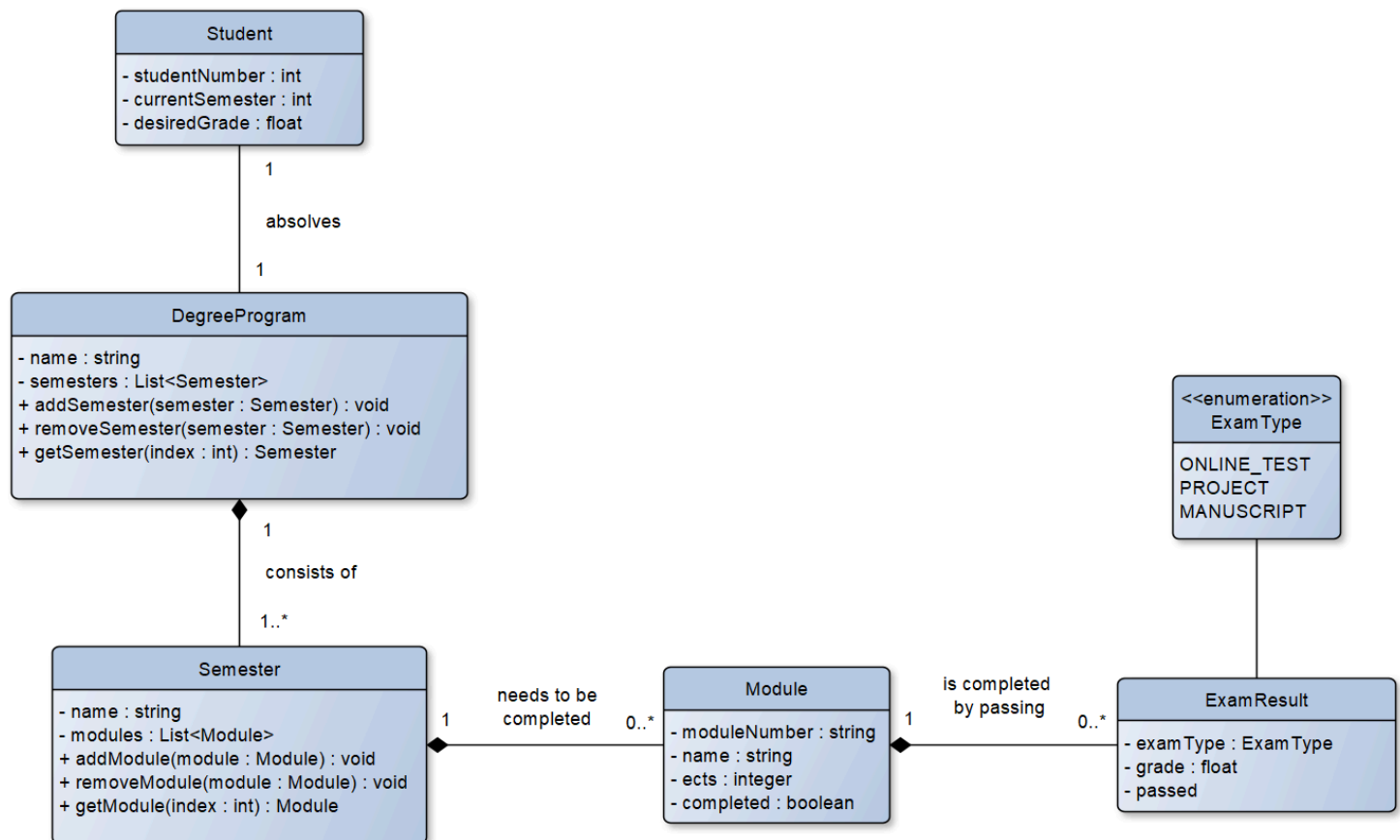
Semester

- aktuelle Module
- Status des Moduls
- Prüfungsleistung
- gegebenenfalls Note

FUNKTIONEN

- Anzahl der Semester festlegen
- aktuelles Semester festlegen
- Modul hinzufügen zu Semester
- Modul bearbeiten
- Modul löschen
- Prüfungsleistung (Note) für Modul eingeben
- Prüfungsleistung (Note) für Modul ändern
- Modul abschließen
- angestrebten Notendurchschnitt festlegen





Student – DegreeProgram → Assoziation

Ein *Student* ist im Rahmen des Dashboards genau einem *DegreeProgram* (Studiengang) zugeordnet. Die Modellierung wird bewusst auf das aktuell betrachtete *DegreeProgram* beschränkt.

DegreeProgram - Semester → Komposition

Die *Semester* sind Bestandteil der Struktur eines *DegreeProgram*. Sie werden innerhalb dieses *DegreeProgram* verwaltet und besitzen in diesem Modell keinen eigenständigen Lebenszyklus. Wird das *DegreeProgram* entfernt, werden auch die zugehörigen *Semester* entfernt. Daher wird eine Komposition verwendet.

Semester - Module → Komposition

Die *Modules* werden im Modell einem *Semester* des *DegreeProgram* zugeordnet und sind Bestandteil dieses *Semesters*. Ein *Module* gehört innerhalb des Modells zu genau einem *Semester* und besitzt keinen eigenständigen Lebenszyklus außerhalb dieses *Semesters*. Daher wird eine Komposition verwendet.

Module - ExamResult → Komposition

Ein *Module* kann keine, eine oder mehrere *ExamResults* (Prüfungsleistungen) besitzen. Ein *ExamResult* gehört dabei immer genau zu einem *Module* und wird innerhalb dieses *Modules* verwaltet. Ein *ExamResult* besitzt in diesem Modell keinen eigenständigen Lebenszyklus außerhalb des zugehörigen *Modules*. Wird ein *Module* gelöscht, werden auch die zugehörigen Prüfungsleistungen gelöscht. Daher wird zwischen *Module* und *ExamResult* eine Komposition verwendet.

Vererbung

Eine Vererbungshierarchie wurde bewusst nicht verwendet, da dieses Projekt eine abstrahierte, stark vereinfachte Version eines "echten Studiengangs" darstellt. Zwischen den modellierten Entity-Klassen besteht keine sinnvolle „ist-ein“-Beziehung, die eine Vererbung rechtfertigen würde. Die verschiedenen Prüfungsarten (*OnlineTest*, *Projekt*, *Manuskript*) werden stattdessen durch das *ExamType*-Enum abgebildet. Eine Unterteilung der Prüfungsleistungen in verschiedene Unterklassen würde das Modell für die Anforderungen des Dashboards unnötig verkomplizieren.

Entwicklungsumgebung	PyCharm	PyCharm ist speziell auf die Entwicklung mit Python ausgerichtet und bietet unter anderem Funktionen wie Syntaxprüfung, Code-Vervollständigung, Debugging und eine übersichtliche Projektverwaltung. Dadurch eignet sich PyCharm besonders gut für die Entwicklung des Dashboards und unterstützt dabei, den Python-Code übersichtlich und strukturiert aufzubauen. Da im weiteren Verlauf des Projekts objektorientierte Programmierung mit mehreren Klassen eingesetzt wird, bietet PyCharm hierfür eine passende Entwicklungsumgebung.
Benutzerinteraktion	Grafische Benutzeroberfläche	Da es sich bei dem Projekt um ein Dashboard handelt, bietet eine GUI die Möglichkeit, die wichtigsten Informationen übersichtlich und auf einen Blick darzustellen.
GUI-Bibliothek	Tkinter	Für die prototypische Umsetzung wird Tkinter verwendet, da es Bestandteil der Standardbibliothek von Python ist und die Erstellung einfacher grafischer Benutzeroberflächen ermöglicht. Für die Anforderungen des Course-Dashboards ist der Funktionsumfang ausreichend, ohne eine zusätzliche umfangreiche GUI-Bibliothek einsetzen zu müssen.
Datenspeicherung	JSON	Die Daten werden für den Prototyp in einer JSON-Datei gespeichert. JSON eignet sich aufgrund seiner einfachen und strukturierten Darstellung für die Speicherung der relativ kleinen Datenmenge. Außerdem kann Python JSON-Dateien ohne zusätzliche Bibliotheken verarbeiten. Der Einsatz einer relationalen Datenbank wäre für den Prototypen aufgrund der überschaubaren Datenmenge nicht erforderlich.
Versionsverwaltung	Git	Git wird zur Versionsverwaltung eingesetzt. Dadurch können Änderungen am Quellcode nachvollzogen und frühere Versionen bei Bedarf wiederhergestellt werden.

JSON-Speicherung

Ein einfaches Script, um das Speichern und Laden von JSON-Daten zu testen.

```
import json

data = {
    "courses": [
        {
            "id": "Kurs001",
            "name": "SQL"
        },
        {
            "id": "Kurs002",
            "name": "Python"
        },
        {
            "id": "Kurs003",
            "name": "SQL Datamart Project"
        },
        {
            "id": "Kurs004",
            "name": "Python Course Dashboard Project"
        }
    ]
}

with open("courses.json", "w", encoding="utf-8") as file:
    json.dump(data, file, indent=4, ensure_ascii=False)

print("courses.json wurde erstellt.")

print("courses.json wird geladen.")

with open("courses.json", "r", encoding="utf-8") as file:
    loaded_data = json.load(file)

print("Inhalt von courses.json:", loaded_data)

courses = loaded_data["courses"]

for course in courses:
    print("Course: ", course)
```

GUI

Einfaches GUI-Beispiel, um Python-Funktionalitäten zu testen.

(Code ist nicht vollständig dargestellt)

```
1 import tkinter as tk
2 from tkinter import ttk
3
4 root = tk.Tk()
5 root.title("Course Dashboard")
6 root.geometry("600x300")
7 root.resizable(width=False, height=False)
8
9 title = tk.Label(
10     root,
11     text="COURSE-DASHBOARD",
12     font=("Arial", 18, "bold")
13 )
14 title.pack(pady=20)
15
16 cards = tk.Frame(root)
17 cards.pack(fill="both", expand=True, padx=20)
18
19 cards.columnconfigure(index=0, weight=1)
20 cards.columnconfigure(index=1, weight=1)
21
22 # STUDIENFortschritt
23 progress_card = tk.LabelFrame(
24     cards,
```

